

بسم الله الرحمن الرحيم



گزارش پروژه کارشناسی

دانشجویان:

زهرا مصومی 9873131

آذین مایار 9873126

استاد راهنما پروژه : مهندس ریحانه دهقان

تابستان 1402

Malaria detection

کد اولیه:

```
[1] import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
from google.colab import drive
from sklearn.metrics import classification_report
import os

# Mount Google Drive
drive.mount('/content/drive')

Mounted at /content/drive

[5] # Set file paths
file_path = '/content/drive/MyDrive/cell_images'
test_path = file_path + "/test/"
train_path = file_path + "/train/"

[6] # Print number of images in each class
print("Number of parasitized images in test set:", len(os.listdir(test_path + "/parasitized/")))
print("Number of uninfected images in test set:", len(os.listdir(test_path + "/uninfected/")))
print("Number of parasitized images in train set:", len(os.listdir(train_path + "/parasitized/")))
print("Number of uninfected images in train set:", len(os.listdir(train_path + "/uninfected/")))

Number of parasitized images in test set: 1300
Number of uninfected images in test set: 1300
Number of parasitized images in train set: 1565
Number of uninfected images in train set: 1279
```

ابتدا ایمپورت های لازم را انجام داده و به google collab دسترسی برای استفاده از dataset می

دهیم

می بینیم که دیتاست شامل 2 دسته test و train با تعداد نشان داده شده، می باشد.

```
[ ] # Create data generator with rescaling
data_generator = ImageDataGenerator(rescale=1./255)
```

```
[ ] # Load training data
train_data = data_generator.flow_from_directory(
    train_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)
```

Found 24998 images belonging to 2 classes.

```
[ ] # Load test data
test_data = data_generator.flow_from_directory(
    test_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)
```

Found 2600 images belonging to 2 classes.

پس از این که دیتای test و train را لود کردیم مدل را با لایه‌های اولیه که مطابق مقاله اولیه است مشخص می‌کنیم:

```
# Define the model architecture
model = tf.keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(2, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

سپس مدل را train می‌کنیم:

```
[ ] # Train the model
model.fit(train_data, validation_data=test_data, epochs=10)
```

```
Epoch 1/10
782/782 [=====] - 3289s 4s/step - loss: 0.4615 - accuracy: 0.7971 - val_loss: 0.6113 - val_accuracy: 0.8150
Epoch 2/10
782/782 [=====] - 263s 337ms/step - loss: 0.2055 - accuracy: 0.9252 - val_loss: 0.1870 - val_accuracy: 0.9358
Epoch 3/10
782/782 [=====] - 253s 324ms/step - loss: 0.1651 - accuracy: 0.9419 - val_loss: 0.7865 - val_accuracy: 0.7592
Epoch 4/10
782/782 [=====] - 253s 324ms/step - loss: 0.1491 - accuracy: 0.9498 - val_loss: 0.2300 - val_accuracy: 0.9288
Epoch 5/10
782/782 [=====] - 262s 335ms/step - loss: 0.1405 - accuracy: 0.9515 - val_loss: 0.1326 - val_accuracy: 0.9531
Epoch 6/10
782/782 [=====] - 253s 323ms/step - loss: 0.1293 - accuracy: 0.9556 - val_loss: 0.2462 - val_accuracy: 0.8981
Epoch 7/10
782/782 [=====] - 265s 338ms/step - loss: 0.1235 - accuracy: 0.9577 - val_loss: 0.1549 - val_accuracy: 0.9381
Epoch 8/10
782/782 [=====] - 255s 326ms/step - loss: 0.1095 - accuracy: 0.9606 - val_loss: 0.1498 - val_accuracy: 0.9523
Epoch 9/10
782/782 [=====] - 262s 334ms/step - loss: 0.1005 - accuracy: 0.9650 - val_loss: 0.1531 - val_accuracy: 0.9465
Epoch 10/10
782/782 [=====] - 267s 342ms/step - loss: 0.1030 - accuracy: 0.9644 - val_loss: 0.1457 - val_accuracy: 0.9542
<keras.callbacks.History at 0x7f1a97561b40>
```

```
[11] # Evaluate the model on the test data
loss, accuracy = model.evaluate(test_data)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
82/82 [=====] - 11s 138ms/step - loss: 0.1457 - accuracy: 0.9542
Test Loss: 0.14565975964069366
Test Accuracy: 0.954230785369873
```

محاسبه f1score و سایر المان ها

```
[10] # Make predictions on the test data
y_pred = model.predict(test_data)
y_pred = np.argmax(y_pred, axis=1) # Convert predicted probabilities to class labels

82/82 [=====] - 11s 138ms/step

[9] # Get true labels
y_true = test_data.classes

# Compute precision, recall, and F1 score
report = classification_report(y_true, y_pred)
print(report)
```

	precision	recall	f1-score	support
0	0.50	0.49	0.50	1300
1	0.50	0.52	0.51	1300
accuracy			0.50	2600
macro avg	0.50	0.50	0.50	2600
weighted avg	0.50	0.50	0.50	2600

برای شروع تغییرات در مدل ابتدا با Data Augmentation شروع می کنیم که دیتای ترین ما را گسترش می دهد و با ایجاد تغییرات متفاوت در تصاویر دیتاست، عمومیت شبکه را بالاتر می برد و برای کاهش اورفیت نیز مفید است.

Option 1: Data Augmentation

[]

```
augmented_data_generator = ImageDataGenerator(  
    rotation_range=20,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2,  
    horizontal_flip=True,  
    vertical_flip=True,  
    fill_mode='nearest'  
)
```



```
train_data_augmented = augmented_data_generator.flow_from_directory(  
    train_path,  
    target_size=(64, 64),  
    batch_size=32,  
    class_mode='categorical'  
)
```



Found 24998 images belonging to 2 classes.

[]

```
model.fit(train_data_augmented, validation_data=test_data, epochs=10)
```

```
Epoch 1/10  
782/782 [=====] - 304s 389ms/step - loss: 0.2596 - accuracy: 0.9105 - val_loss: 0.6929 - val_accuracy: 0.5027  
Epoch 2/10  
782/782 [=====] - 297s 379ms/step - loss: 0.2400 - accuracy: 0.9177 - val_loss: 0.7001 - val_accuracy: 0.5000  
Epoch 3/10  
782/782 [=====] - 290s 371ms/step - loss: 0.2329 - accuracy: 0.9214 - val_loss: 0.7110 - val_accuracy: 0.5000  
Epoch 4/10  
782/782 [=====] - 277s 354ms/step - loss: 0.2236 - accuracy: 0.9250 - val_loss: 0.7301 - val_accuracy: 0.5000  
Epoch 5/10  
782/782 [=====] - 288s 368ms/step - loss: 0.2227 - accuracy: 0.9238 - val_loss: 0.7213 - val_accuracy: 0.5000  
Epoch 6/10  
782/782 [=====] - 289s 369ms/step - loss: 0.2264 - accuracy: 0.9239 - val_loss: 0.7900 - val_accuracy: 0.5000  
Epoch 7/10  
782/782 [=====] - 286s 365ms/step - loss: 0.2217 - accuracy: 0.9242 - val_loss: 0.8162 - val_accuracy: 0.5000  
Epoch 8/10  
782/782 [=====] - 295s 377ms/step - loss: 0.2160 - accuracy: 0.9268 - val_loss: 0.7580 - val_accuracy: 0.5000  
Epoch 9/10  
782/782 [=====] - 283s 361ms/step - loss: 0.2149 - accuracy: 0.9265 - val_loss: 0.7468 - val_accuracy: 0.5000  
Epoch 10/10  
782/782 [=====] - 297s 380ms/step - loss: 0.2160 - accuracy: 0.9262 - val_loss: 0.8276 - val_accuracy: 0.5000  
<keras.callbacks.History at 0x7f1a973da590>
```

[]

متوجه می شویم که شبکه دچار اورفیت شده است چرا که با وجود بالا رفتن اکیورسی، ولیدیشن اکیورسی پایین است.

برای مرحله بعدی تصمیم گرفتیم که لایه های dense را افزایش دهیم تا بتوان عملیات بیشتری در مدل انجام داد

لایه ها را صورت زیر تغییر دادیم:

ابتدا دیتا را rescale کردیم (برای اعمال data augmentation)

```
[11]
# Load the data using ImageDataGenerator
data_generator = ImageDataGenerator(rescale=1./255)

train_data = data_generator.flow_from_directory(
    train_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)

test_data = data_generator.flow_from_directory(
    test_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)

Found 24998 images belonging to 2 classes.
Found 2600 images belonging to 2 classes.
```

و سپس تغییرات را ایجاد کردیم:

```

# Define the model architecture
model = tf.keras.Sequential([
    layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64, 64, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Conv2D(256, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(2, activation='softmax')
])

```

پس از کامپایل، مدل خود را train کردیم و نتیجه به صورت زیر شد:

```

# Evaluate the model on the test data
loss, accuracy = model.evaluate(test_data)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

```

```

82/82 [=====] - 28s 340ms/step - loss: 0.1157 - accuracy: 0.9577
Test Loss: 0.11568818241357803
Test Accuracy: 0.9576923251152039

```

```

[ ]
# Retrieve the accuracy from the training history
test_accuracy = history.history['val_accuracy']
print("Test Accuracy from History:", test_accuracy[-1])

```

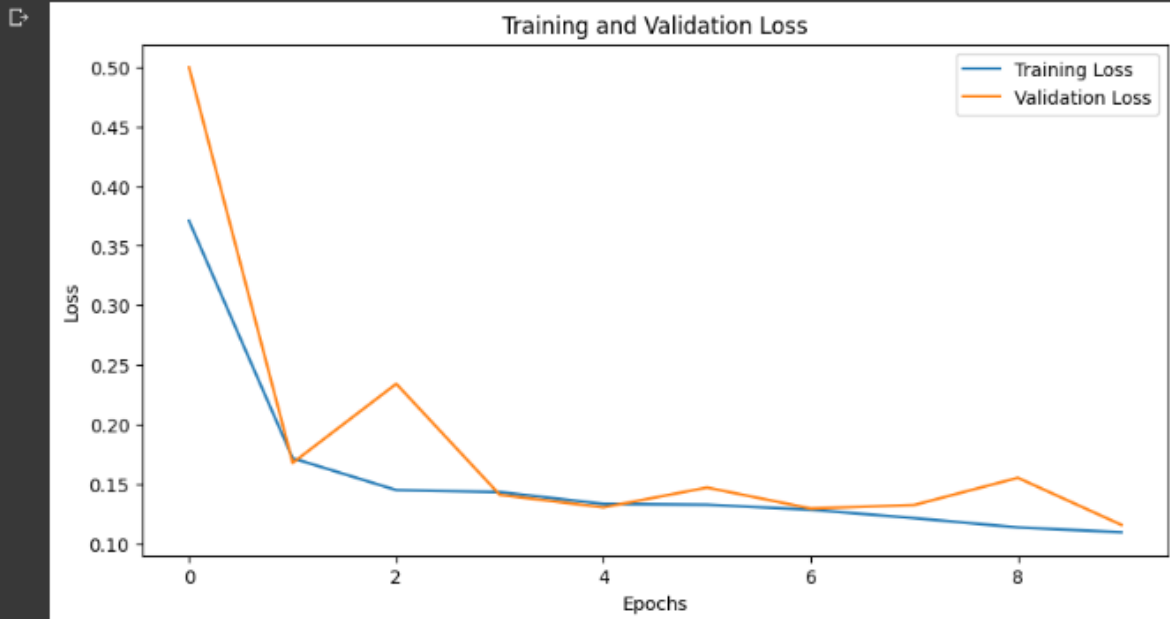
```

Test Accuracy from History: 0.9576923251152039

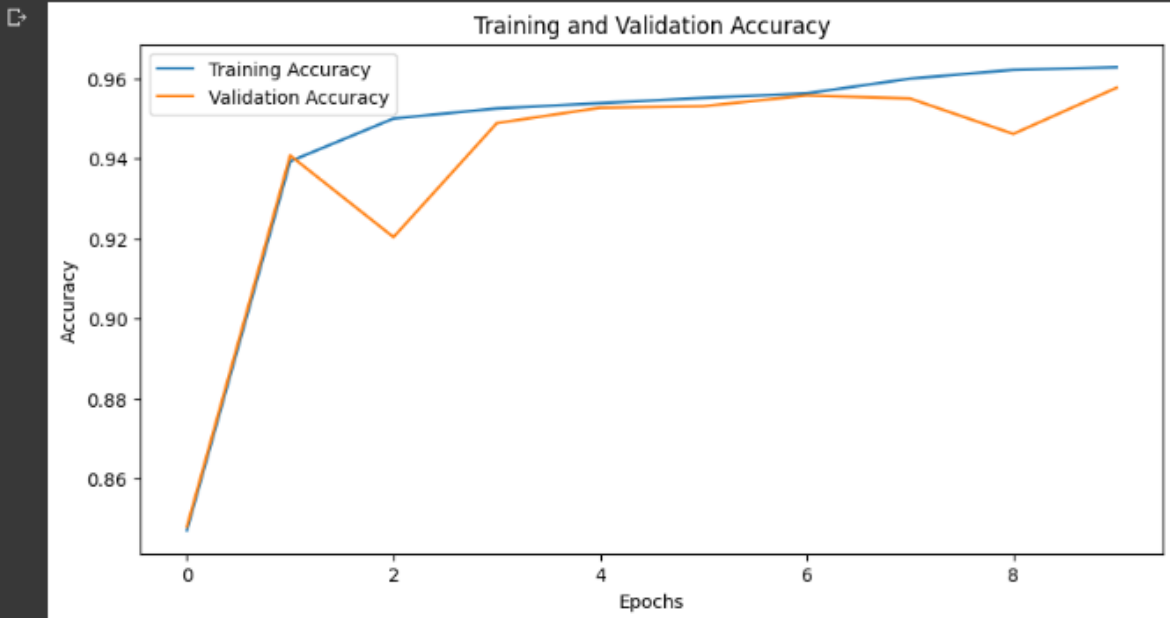
```

نمودارها زیر به شکل زیر هستند:

```
# Plot the training and validation loss
plt.figure(figsize=(10, 5))
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.show()
```



```
# Plot the training and validation accuracy
plt.figure(figsize=(10, 5))
plt.plot(train_acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.show()
```



در تلاش بعدی با توجه به نتیجه مطلوب نمونه قبلی، حال با توجه به دیتاستی که داریم تعداد ایپاک های مدل خود را بالا می بریم تا کارکرد شبکه بهتر مشخص شود.

البته برای رخ ندادن overfit با توجه به بالا بودن تعداد ایپاک ها، از early stopping استفاده می کنیم که در صورت عدم پیشرفت در train، پروسه را متوقف کرده و بهترین وزن ها را مطابق کد زیر ذخیره می کند.

```
[ ] # Set up callbacks for early stopping and learning rate scheduling
early_stopping = tf.keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True)
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(factor=0.1, patience=2)
```

پروسه train در ایپاک 11م متوقف می شود:

```
[10] # Train the model with data augmentation and callbacks
history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=50,
    callbacks=[early_stopping, lr_scheduler]
)

Epoch 1/50
782/782 [=====] - 4285s 5s/step - loss: 0.3483 - accuracy: 0.8553 - val_loss: 0.2298 - val_accuracy: 0.9077 - lr: 0.0010
Epoch 2/50
782/782 [=====] - 740s 946ms/step - loss: 0.1653 - accuracy: 0.9425 - val_loss: 0.1819 - val_accuracy: 0.9438 - lr: 0.0010
Epoch 3/50
782/782 [=====] - 716s 916ms/step - loss: 0.1473 - accuracy: 0.9490 - val_loss: 0.1542 - val_accuracy: 0.9508 - lr: 0.0010
Epoch 4/50
782/782 [=====] - 731s 934ms/step - loss: 0.1419 - accuracy: 0.9515 - val_loss: 0.1191 - val_accuracy: 0.9581 - lr: 0.0010
Epoch 5/50
782/782 [=====] - 750s 959ms/step - loss: 0.1342 - accuracy: 0.9554 - val_loss: 0.1401 - val_accuracy: 0.9515 - lr: 0.0010
Epoch 6/50
782/782 [=====] - 728s 930ms/step - loss: 0.1306 - accuracy: 0.9561 - val_loss: 0.1419 - val_accuracy: 0.9581 - lr: 0.0010
Epoch 7/50
782/782 [=====] - 728s 931ms/step - loss: 0.1047 - accuracy: 0.9644 - val_loss: 0.1171 - val_accuracy: 0.9596 - lr: 1.0000e-04
Epoch 8/50
782/782 [=====] - 743s 950ms/step - loss: 0.0968 - accuracy: 0.9669 - val_loss: 0.1144 - val_accuracy: 0.9615 - lr: 1.0000e-04
Epoch 9/50
782/782 [=====] - 721s 922ms/step - loss: 0.0929 - accuracy: 0.9677 - val_loss: 0.1163 - val_accuracy: 0.9615 - lr: 1.0000e-04
Epoch 10/50
782/782 [=====] - 719s 920ms/step - loss: 0.0900 - accuracy: 0.9691 - val_loss: 0.1222 - val_accuracy: 0.9585 - lr: 1.0000e-04
Epoch 11/50
782/782 [=====] - 718s 918ms/step - loss: 0.0804 - accuracy: 0.9723 - val_loss: 0.1185 - val_accuracy: 0.9600 - lr: 1.0000e-05
```

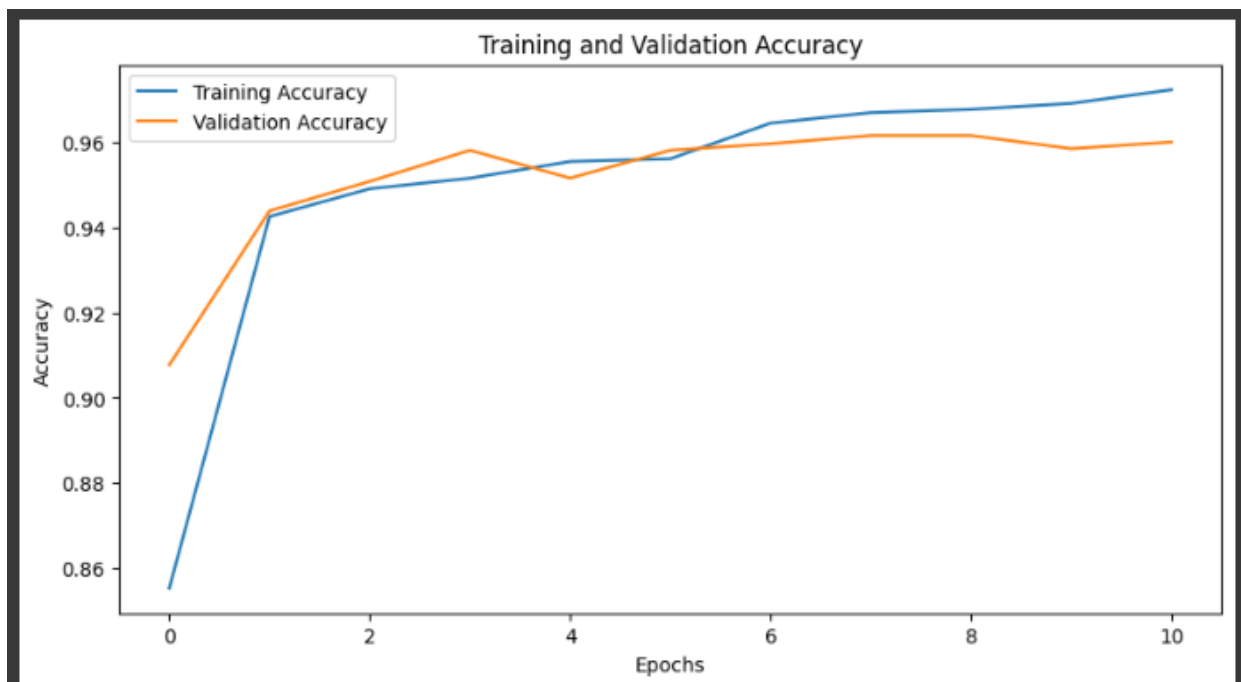
و نتایج زیر را باقی می گذارد:

```
# Evaluate the model on the test data
loss, accuracy = model.evaluate(test_data)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

82/82 [=====] - 21s 258ms/step - loss: 0.1144 - accuracy: 0.9615
Test Loss: 0.11442390084266663
Test Accuracy: 0.9615384340286255
```

✓ **ACCURACY: 96.15%**

✓ **LOSS: 0.1**



به عنوان قدم بعدی تصمیم گرفتیم learning rate scheduling را امتحان کنیم.

برای این کار ابتدا تابعی می نویسیم که پس از اپاک 10م نرخ لرنینگ را کم کند:

```
def lr_schedule(epoch):  
    initial_learning_rate = 0.001  
    decay_steps = 10000  
    decay_rate = 0.9  
    return initial_learning_rate * decay_rate**(epoch // decay_steps)
```

و آن را صدا می زنیم:

```
▶# Train the model with data augmentation and callbacks  
history = model.fit(  
    train_data,  
    validation_data=test_data,  
    epochs=50,  
    callbacks=[early_stopping,tf.keras.callbacks.LearningRateScheduler(lr_scheduler)]  
)
```

نتیجه: در ایپاک پنجم با اکیورسی 95 درصد متوقف شد.

در مرحله بعدی تصمیم گرفتیم در لایه های مدل از batch normalization استفاده کنیم :

در بخش import ها از Tensorflow، BatchNormalization را ایمپورت می کنیم تا در مدل از آن استفاده کنیم:

```
[ ] import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
import os
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
from tensorflow.keras.callbacks import LearningRateScheduler
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.layers import BatchNormalization
```



```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```



Mounted at /content/drive

دیتاست را mount کرده و پس از تخصیص دیتا به test و train آن را load می کنیم:

```
[ ] # Set the file paths for train and test data
file_path = '/content/drive/MyDrive/cell_images'
test_path = file_path + "/test/"
train_path = file_path + "/train/"

# Load the data using ImageDataGenerator
data_generator = ImageDataGenerator(rescale=1./255)

train_data = data_generator.flow_from_directory(
    train_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)

test_data = data_generator.flow_from_directory(
    test_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)

Found 24998 images belonging to 2 classes.
Found 2600 images belonging to 2 classes.
```

استفاده از early stopping برای جلوگیری از overfit کردن شبکه:

```
[ ] # a function for learning rate scheduling that reduces the learning rate exponentially after the 10th epoch

def lr_schedule(epoch):
    initial_learning_rate = 0.001
    decay_steps = 10000
    decay_rate = 0.9
    return initial_learning_rate * decay_rate**(epoch // decay_steps)
```

و در نهایت اضافه کردن batch normalization در هر لایه:

```

model = tf.keras.Sequential([
    layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64, 64, 3)),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Conv2D(256, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.2),

    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(2, activation='softmax')
])

```

کامپایل و train کردن:

```

[ ] # Set up callbacks for early stopping and learning rate scheduling
early_stopping = tf.keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True)

[ ] # Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

[ ] # Train the model with data augmentation and callbacks
history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=50,
    callbacks=[early_stopping, tf.keras.callbacks.LearningRateScheduler(lr_schedule)]
)

```

نتیجه train به ما نشان می دهد که پروسه train در اپیاک 8م با مضخصات زیر متوقف شده است:

loss: 0.1180

accuracy: 0.9601

val_loss: 0.1562

val_accuracy: 0.9558

```
Epoch 1/50
782/782 [=====] - 4782s 6s/step - loss: 0.1657 - accuracy: 0.9427 - val_loss: 0.3788 - val_accuracy: 0.8923 - lr: 0.0010
Epoch 2/50
782/782 [=====] - 913s 1s/step - loss: 0.1524 - accuracy: 0.9478 - val_loss: 0.1602 - val_accuracy: 0.9500 - lr: 0.0010
Epoch 3/50
782/782 [=====] - 932s 1s/step - loss: 0.1439 - accuracy: 0.9517 - val_loss: 0.1380 - val_accuracy: 0.9550 - lr: 0.0010
Epoch 4/50
782/782 [=====] - 916s 1s/step - loss: 0.1384 - accuracy: 0.9525 - val_loss: 0.2268 - val_accuracy: 0.9223 - lr: 0.0010
Epoch 5/50
782/782 [=====] - 912s 1s/step - loss: 0.1339 - accuracy: 0.9541 - val_loss: 0.1375 - val_accuracy: 0.9562 - lr: 0.0010
Epoch 6/50
782/782 [=====] - 924s 1s/step - loss: 0.1304 - accuracy: 0.9554 - val_loss: 0.1766 - val_accuracy: 0.9396 - lr: 0.0010
Epoch 7/50
782/782 [=====] - 906s 1s/step - loss: 0.1270 - accuracy: 0.9576 - val_loss: 0.1661 - val_accuracy: 0.9442 - lr: 0.0010
Epoch 8/50
782/782 [=====] - 911s 1s/step - loss: 0.1180 - accuracy: 0.9601 - val_loss: 0.1562 - val_accuracy: 0.9558 - lr: 0.0010
```

نتیجه اکیورسی روی دیتای تست هم به صورت زیر است:



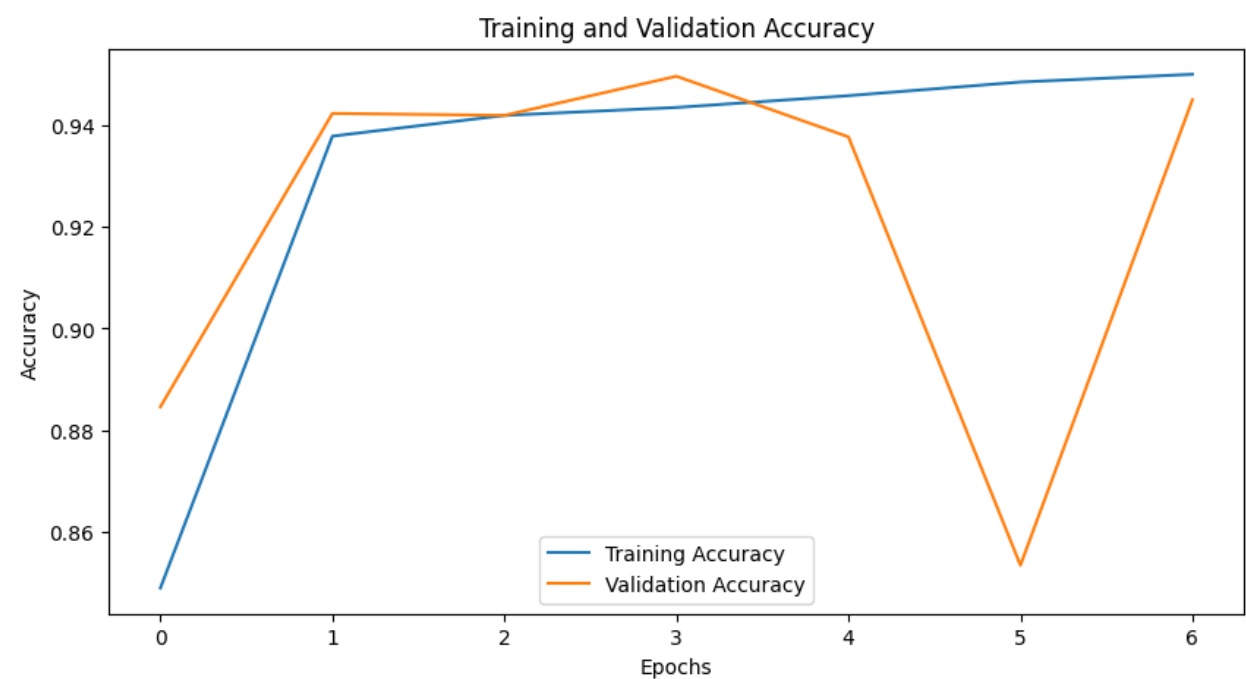
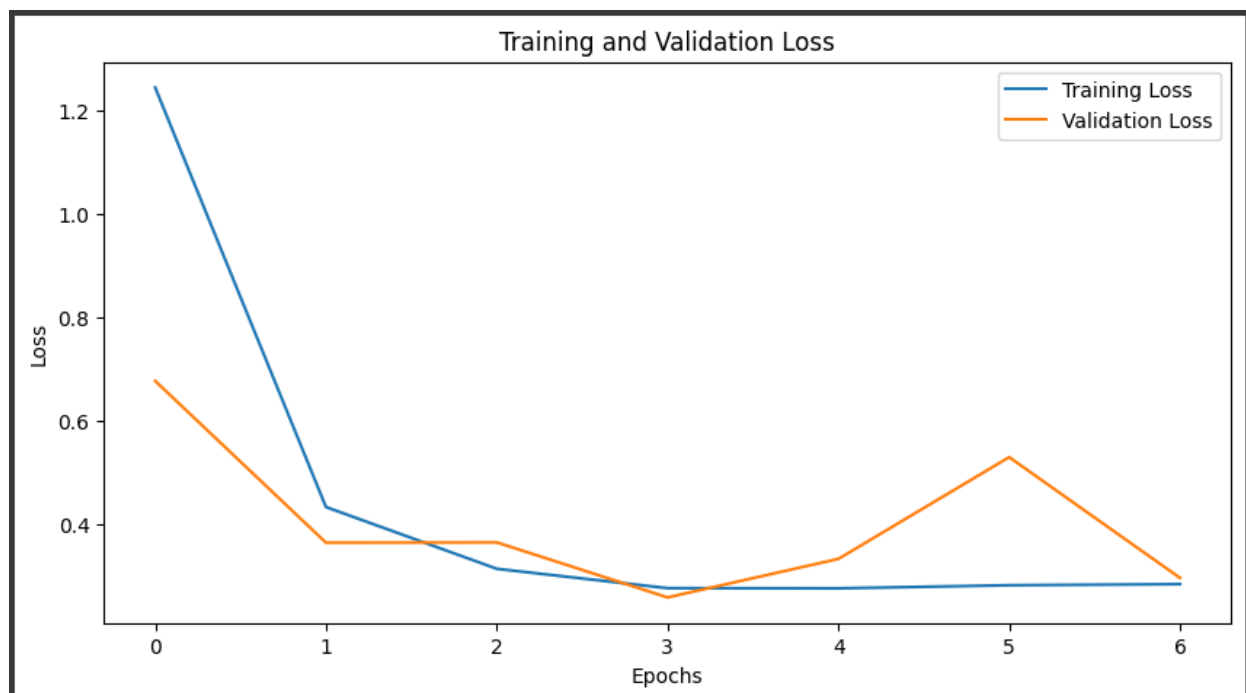
```
# Evaluate the model on the test data
loss, accuracy = model.evaluate(test_data)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
82/82 [=====] - 23s 279ms/step - loss: 0.2589 - accuracy: 0.9496
Test Loss: 0.25894010066986084
Test Accuracy: 0.9496153593063354
```

Test Loss: 0.25

Test Accuracy: 0.949

نمودارها:



برای قدم بعدی از L2 regularization بهره می بریم:

ابتدا ایمپورت های لازم را انجام می دهیم:

```
[ ] import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
import os
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report
from tensorflow.keras.callbacks import LearningRateScheduler
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.regularizers import l2
```



```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
```



Mounted at /content/drive

دسته بندی دیتا به تست و ترین و لود کردن آن ها از دیتاست:

```
# Set the file paths for train and test data
file_path = '/content/drive/MyDrive/cell_images'
test_path = file_path + "/test/"
train_path = file_path + "/train/"

[ ] # Load the data using ImageDataGenerator
data_generator = ImageDataGenerator(rescale=1./255)

train_data = data_generator.flow_from_directory(
    train_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)

test_data = data_generator.flow_from_directory(
    test_path,
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical'
)

Found 24998 images belonging to 2 classes.
Found 2600 images belonging to 2 classes.
```

مطابق شکل در ساختار مدا از regularization استفاده میکنیم:

```

model = tf.keras.Sequential([
    layers.Conv2D(64, (3, 3), activation='relu', input_shape=(64, 64, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Conv2D(256, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Flatten(),
    layers.Dense(512, activation='relu', kernel_regularizer=l2(0.001)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(256, activation='relu', kernel_regularizer=l2(0.001)),
    layers.BatchNormalization(),
    layers.Dropout(0.2),

    layers.Dense(2, activation='softmax')
])

```

سپس مدل را کامپایل و در نهایت train می کنیم:

```

[ ] # Set up callbacks for early stopping and learning rate scheduling
early_stopping = tf.keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True)

[ ] # Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

[ ] # Train the model with data augmentation and callbacks
history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=50,
    callbacks=[early_stopping, tf.keras.callbacks.LearningRateScheduler(lr_schedule)]
)

```

نتیجه train شبکه:

```
Epoch 1/50  
782/782 [=====] - 6584s 8s/step - loss: 1.2449 - accuracy: 0.8490 - val_loss: 0.6776 - val_accuracy: 0.8846 - lr: 0.0010  
Epoch 2/50  
782/782 [=====] - 744s 951ms/step - loss: 0.4337 - accuracy: 0.9378 - val_loss: 0.3648 - val_accuracy: 0.9423 - lr: 0.0010  
Epoch 3/50  
782/782 [=====] - 715s 915ms/step - loss: 0.3143 - accuracy: 0.9419 - val_loss: 0.3653 - val_accuracy: 0.9419 - lr: 0.0010  
Epoch 4/50  
782/782 [=====] - 725s 926ms/step - loss: 0.2768 - accuracy: 0.9435 - val_loss: 0.2589 - val_accuracy: 0.9496 - lr: 0.0010  
Epoch 5/50  
782/782 [=====] - 725s 927ms/step - loss: 0.2765 - accuracy: 0.9458 - val_loss: 0.3334 - val_accuracy: 0.9377 - lr: 0.0010  
Epoch 6/50  
782/782 [=====] - 720s 921ms/step - loss: 0.2825 - accuracy: 0.9485 - val_loss: 0.5302 - val_accuracy: 0.8535 - lr: 0.0010  
Epoch 7/50  
782/782 [=====] - 708s 905ms/step - loss: 0.2846 - accuracy: 0.9500 - val_loss: 0.2967 - val_accuracy: 0.9450 - lr: 0.0010
```

می بینیم که مدل با اکیورسی 95 در ایپاک 7 متوقف شده است.

Loss: 0.28

Accuracy: 0.95

Validation loss: 0.29

Validation accuracy 94.5

نتیجه استفاده از regularization باعث پایین آمدن accuracy نسبت به مدل های قبلی و بالا رفتن loss شد پس نتیجه قابل قبولی برای ما نداشت.

نتیجه نهایی:

مطابق تغییراتی که در شبکه ایجاد شد بهترین نتیجه با اعمال Data Augmentation حاصل شد که مدل ما را به اکیورسی 96 به بالا رساند.

```
# Evaluate the model on the test data  
loss, accuracy = model.evaluate(test_data)  
print("Test Loss:", loss)  
print("Test Accuracy:", accuracy)  
  
82/82 [=====] - 21s 258ms/step - loss: 0.1144 - accuracy: 0.9615  
Test Loss: 0.11442390084266663  
Test Accuracy: 0.9615384340286255
```

✓ **ACCURACY: 96.15%**

✓ **LOSS: 0.1**